# Compte rendu de TP

## 1. Introduction

L'objectif de ce TP est de développer en Kotlin une application Android « Currency Converter ». Elle convertit un montant d'une devise vers une autre à partir de taux fixes, grâce à un clavier numérique personnalisé. L'application utilise un ConstraintLayout, un TableLayout, deux Spinner, le View Binding et un Toast pour les erreurs de saisie.

## 2. Choix d'interface

- **ConstraintLayout principal :** un Guideline horizontal placé à 45 % de la hauteur sépare l'écran en deux zones. Cela donne la même proportion sur tous les écrans.
- **Zone du haut :** un View turquoise (#2BB8E8) sert de fond. Il contient le titre, les deux Spinner, le montant et le résultat (TextView de 48sp alignés à droite et soulignés), le bouton ⇅ et le bouton Convertir.
- **Zone du bas :** un TableLayout de 4 TableRow × 3 colonnes. Chaque ligne et chaque touche utilisent layout_weight=1 pour occuper tout l'espace de façon égale. Les chiffres sont en #1BA6C9 et la touche C en #F0727A, avec le style borderlessButtonStyle et 32sp.
- **Saisie :** le montant et le résultat sont des TextView, donc le clavier système n'apparaît jamais. Seul le clavier de l'application permet de saisir.
- **Aucune valeur en dur :** les textes sont dans strings.xml (avec le string-array devises) et les couleurs dans colors.xml. L'orientation est bloquée en portrait dans le manifeste.
- **Identifiants :** tvTitre, spDeviseSource, tvMontant, btnInverser, spDeviseCible, tvResultat, btnConvertir.

## 3. Logique de conversion

Les taux sont fixes et exprimés pour 1 EUR. Ils sont stockés dans une Map<String, Double>. La conversion passe par l'euro, qui sert de devise pivot :

```
montant en EUR = montant / taux(source)
résultat       = montant en EUR × taux(cible)
```

Exemple avec 120 USD vers EUR : 120 / 1.17 = 102.5641…, soit 102.56 une fois arrondi. Le résultat est affiché avec String.format(Locale.US, "%.2f", …). Locale.US garantit un point décimal même si le téléphone est réglé en français.

```
private fun convertir(montant: Double, source: String, cible: String): Double {
    val montantEnEuro = montant / taux.getValue(source)
    return montantEnEuro * taux.getValue(cible)
}
```

**Règles de gestion.** La fonction ajouterCaractere(c) ignore un deuxième point et refuse tout caractère au-delà de 10. Le « 0 » initial est remplacé par le premier chiffre tapé. La touche C remet le montant à 0 et vide le résultat. Si le montant vaut 0, un Toast « Veuillez saisir un montant » s'affiche. Si la source et la cible sont identiques, le calcul redonne naturellement le montant, car le taux se simplifie.

**Inversion.** Le bouton ⇅ échange les positions des deux Spinner avec setSelection, puis relance la conversion si un résultat est déjà affiché.

## 4. Tests

Les quatre cas du sujet ont été vérifiés. Les valeurs obtenues correspondent aux valeurs attendues.

| Montant | Source | Cible | Attendu |
|---|---|---|---|
| 120 | USD | EUR | 102.56 |
| 100 | EUR | TND | 340.00 |
| 50 | GBP | USD | 67.24 |
| 10 | EUR | EUR | 10.00 |

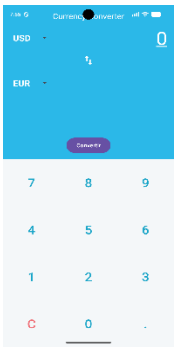



**Figure 1 : écran au démarrage**

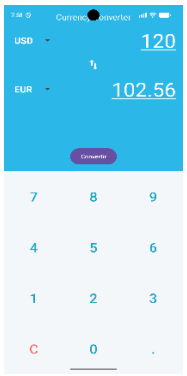



**Figure 2 : 120 USD → EUR = 102.56**

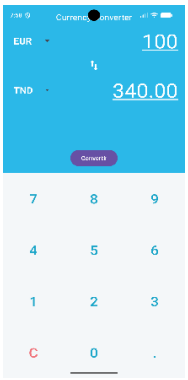



**Figure 3 : 100 EUR → TND = 340.00**

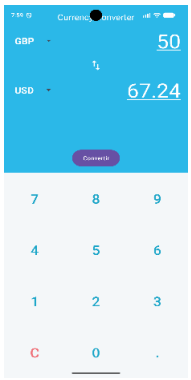



**Figure 4 : 50 GBP → USD = 67.24**

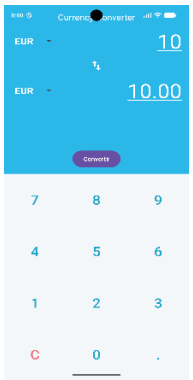



**Figure 5 : 10 EUR → EUR = 10.00**

## 5. Difficultés rencontrées

- **Chaînes manquantes dans strings.xml :** des erreurs « Cannot resolve symbol '@string/touche_X' » sont apparues, car chaque touche du clavier a besoin de son propre string. Elles ont été corrigées en déclarant touche_0 à touche_9.
- **Erreur « Unresolved reference 'main' » :** le code généré par le template cherchait R.id.main, qui n'existe plus dans le nouveau layout. Il a fallu remplacer ce code par l'initialisation du View Binding.
- **Erreurs @+id non résolues :** ces erreurs venaient d'un cache de l'IDE pas à jour. Elles ont disparu après la synchronisation Gradle et la reconstruction du projet (Clean puis Assemble).
- **Séparateur décimal :** sans Locale.US, le formatage pouvait afficher une virgule selon la langue du téléphone.

## 6. Conclusion

L'application respecte le cahier des charges : interface fidèle à la maquette, deux Spinner remplis depuis strings.xml, clavier personnalisé, conversion arrondie à 2 décimales via l'euro, inversion des devises et gestion des erreurs avec un Toast. Des améliorations sont possibles, comme la conversion automatique à chaque saisie, l'utilisation de taux réels via une API (Retrofit) ou la conservation de l'état après rotation avec un ViewModel.